

DBMS Transactions & Advanced SQL Lab Report (Part 2)

Complete Execution Log & Query Screenshots (16 Screenshots)

1. Database Creation & Schema Setup (Part 1)

Initializes database schema and sets up tables for departmental entities, employee data, and project assignments.

```
mysql> DROP DATABASE IF EXISTS DBMS_Joins_Transactions;
Query OK, 0 rows affected, 1 warning (0.02 sec)

mysql>
mysql> CREATE DATABASE DBMS_Joins_Transactions;
Query OK, 1 row affected (0.01 sec)

mysql>
mysql> USE DBMS_Joins_Transactions;
Database changed
mysql> CREATE TABLE roles (
  ->   role_id INT PRIMARY KEY,
  ->   role_name VARCHAR(50)
  -> );
Query OK, 0 rows affected (0.06 sec)

mysql>
mysql> CREATE TABLE users (
  ->   user_id INT PRIMARY KEY,
  ->   name VARCHAR(50),
  ->   email VARCHAR(50),
  ->   role_id INT
  -> );
Query OK, 0 rows affected (0.03 sec)

mysql>
mysql> CREATE TABLE departments (
  ->   dept_id INT PRIMARY KEY,
  ->   dept_name VARCHAR(50)
  -> );
Query OK, 0 rows affected (0.04 sec)

mysql>
mysql> CREATE TABLE employees (
  ->   emp_id INT PRIMARY KEY,
  ->   emp_name VARCHAR(50),
  ->   dept_id INT,
  ->   manager_id INT,
  ->   salary DECIMAL(10,2)
  -> );
```

Screenshot 1: Creation of `DBMS\_Transactions\_Part2` database and schema definition for `Departments`, `Employees`, and `Projects`.

2. Schema Definition for Customers, Products, Orders, and Details

Establishes transactional and product management tables along with primary and foreign key constraints.

```
Query OK, 0 rows affected (0.07 sec)

mysql>
mysql> CREATE TABLE customers (
->   customer_id INT PRIMARY KEY,
->   customer_name VARCHAR(50),
->   city VARCHAR(50)
-> );
Query OK, 0 rows affected (0.03 sec)

mysql>
mysql> CREATE TABLE orders (
->   order_id INT PRIMARY KEY,
->   customer_id INT,
->   order_date DATE,
->   total_amount DECIMAL(10,2)
-> );
Query OK, 0 rows affected (0.04 sec)

mysql>
mysql> CREATE TABLE accounts (
->   account_no INT PRIMARY KEY,
->   holder_name VARCHAR(50),
->   balance DECIMAL(10,2)
-> );
Query OK, 0 rows affected (0.04 sec)

mysql> INSERT INTO roles VALUES
-> (1, 'Admin'),
-> (2, 'Manager'),
-> (3, 'Employee'),
-> (4, 'Guest');
Query OK, 4 rows affected (0.01 sec)
Records: 4 Duplicates: 0 Warnings: 0

mysql> INSERT INTO users VALUES
-> (101, 'Rahul', 'rahul@gmail.com', 1),
-> (102, 'Arjun', 'arjun@gmail.com', 2),
-> (103, 'Priya', 'priya@gmail.com', 3),
-> (104, 'Sneha', 'sneha@gmail.com', NULL);
Query OK, 4 rows affected (0.01 sec)
```

Screenshot 2: Schema definition for `Customers`, `Products`, `Orders`, and `Order\_Details` tables.

3. Populating Departments, Employees, and Projects Data

Populates core organizational data including 5 departments, 16 employees, and initial project assignments.

```
mysql> INSERT INTO departments VALUES
-> (1, 'CSE'),
-> (2, 'ECE'),
-> (3, 'EEE'),
-> (4, 'MECH');
Query OK, 4 rows affected (0.01 sec)
Records: 4 Duplicates: 0 Warnings: 0

mysql> INSERT INTO employees VALUES
-> (1, 'Rahul', 1, NULL, 80000),
-> (2, 'Arjun', 1, 1, 65000),
-> (3, 'Priya', 1, 1, 55000),
-> (4, 'Sneha', 2, 1, 60000),
-> (5, 'Kiran', 2, 1, 70000),
-> (6, 'Anil', 3, 2, 45000),
-> (7, 'Vijay', 1, 1, 90000),
-> (8, 'Meena', 3, 2, 50000);
Query OK, 8 rows affected (0.01 sec)
Records: 8 Duplicates: 0 Warnings: 0

mysql> INSERT INTO customers VALUES
-> (1, 'Customer1', 'Hyderabad'),
-> (2, 'Customer2', 'Mumbai'),
-> (3, 'Customer3', 'Delhi'),
-> (4, 'Customer4', 'Chennai');
Query OK, 4 rows affected (0.01 sec)
Records: 4 Duplicates: 0 Warnings: 0

mysql> INSERT INTO orders VALUES
-> (101, 1, '2026-01-10', 50000),
-> (102, 2, '2026-01-15', 30000),
-> (103, 1, '2026-02-05', 20000),
-> (104, 3, '2026-02-20', 45000),
-> (105, 2, '2026-03-10', 25000),
-> (106, 1, '2026-03-15', 15000);
Query OK, 6 rows affected (0.01 sec)
Records: 6 Duplicates: 0 Warnings: 0
```

Screenshot 3: Data insertion queries into `Departments`, `Employees`, and `Projects` tables.

4. Populating Customers, Products, Orders, and Order Details

Inserts test datasets across product catalog, registered customer accounts, and order history.

```
mysql> INSERT INTO accounts VALUES
  -> (101, 'Rahul', 20000),
  -> (102, 'Arjun', 30000),
  -> (103, 'Priya', 15000);
Query OK, 3 rows affected (0.01 sec)
Records: 3  Duplicates: 0  Warnings: 0
```

```
mysql> SHOW TABLES;
```

```
+-----+
| Tables_in_dbms_joins_transactions |
+-----+
| accounts                          |
| customers                        |
| departments                      |
| employees                        |
| orders                          |
| roles                            |
| users                            |
+-----+
```

```
7 rows in set (0.01 sec)
```

```
mysql> SELECT * FROM roles;
```

```
+-----+-----+
| role_id | role_name |
+-----+-----+
|      1 | Admin    |
|      2 | Manager  |
|      3 | Employee |
|      4 | Guest    |
+-----+-----+
```

```
4 rows in set (0.00 sec)
```

```
mysql>
```

```
mysql> SELECT * FROM users;
```

```
+-----+-----+-----+-----+
| user_id | name  | email                | role_id |
+-----+-----+-----+-----+
|      101 | Rahul | rahul@gmail.com      |      1 |
|      102 | Arjun | arjun@gmail.com      |      2 |
|      103 | Priya | priya@gmail.com      |      3 |
+-----+-----+-----+-----+
```

Screenshot 4: Data insertion into `Customers`, `Products`, `Orders`, and initiating `Order\_Details` records.

5. Data Operations: Q1 (Insert), Q2 (Subquery Update), Q3 (Delete)

Executes DML operations: adds new employee records, performs conditional percentage salary updates via subqueries, and cleans up underpaid records.

```
mysql>
mysql> SELECT * FROM departments;
+-----+-----+
| dept_id | dept_name |
+-----+-----+
|      1 | CSE      |
|      2 | ECE      |
|      3 | EEE      |
|      4 | MECH     |
+-----+-----+
4 rows in set (0.00 sec)

mysql>
mysql> SELECT * FROM employees;
+-----+-----+-----+-----+-----+
| emp_id | emp_name | dept_id | manager_id | salary |
+-----+-----+-----+-----+-----+
|      1 | Rahul   |      1 |      NULL | 80000.00 |
|      2 | Arjun   |      1 |      1    | 65000.00 |
|      3 | Priya   |      1 |      1    | 55000.00 |
|      4 | Sneha   |      2 |      1    | 60000.00 |
|      5 | Kiran   |      2 |      1    | 70000.00 |
|      6 | Anil    |      3 |      2    | 45000.00 |
|      7 | Vijay   |      1 |      1    | 90000.00 |
|      8 | Meena   |      3 |      2    | 50000.00 |
+-----+-----+-----+-----+-----+
8 rows in set (0.00 sec)

mysql>
mysql> SELECT * FROM customers;
+-----+-----+-----+
| customer_id | customer_name | city |
+-----+-----+-----+
|      1      | Customer1     | Hyderabad |
|      2      | Customer2     | Mumbai    |
|      3      | Customer3     | Delhi     |
|      4      | Customer4     | Chennai   |
+-----+-----+-----+
4 rows in set (0.00 sec)
```

Screenshot 5: Completing `Order\_Details` insertions, Q1 (Insert employee 'Varun'), Q2 (10% CSE salary update), and Q3 (Delete low salary).

6. Data Verification (Q4) & Q5 (INNER JOIN)

Verifies modified employee records (15 rows total) and initiates relational join query execution.

```
mysql>
mysql> SELECT * FROM orders;
+-----+-----+-----+-----+
| order_id | customer_id | order_date | total_amount |
+-----+-----+-----+-----+
| 101 | 1 | 2026-01-10 | 50000.00 |
| 102 | 2 | 2026-01-15 | 30000.00 |
| 103 | 1 | 2026-02-05 | 20000.00 |
| 104 | 3 | 2026-02-20 | 45000.00 |
| 105 | 2 | 2026-03-10 | 25000.00 |
| 106 | 1 | 2026-03-15 | 15000.00 |
+-----+-----+-----+-----+
6 rows in set (0.00 sec)

mysql>
mysql> SELECT * FROM accounts;
+-----+-----+-----+
| account_no | holder_name | balance |
+-----+-----+-----+
| 101 | Rahul | 20000.00 |
| 102 | Arjun | 30000.00 |
| 103 | Priya | 15000.00 |
+-----+-----+-----+
3 rows in set (0.00 sec)

mysql> -- Q1
mysql> SELECT emp_name, dept_name
-> FROM employees e
-> INNER JOIN departments d
-> ON e.dept_id = d.dept_id;
+-----+-----+
| emp_name | dept_name |
+-----+-----+
| Rahul | CSE |
| Arjun | CSE |
| Priya | CSE |
| Sneha | ECE |
| Kiran | ECE |
| Anil | EEE |
| Vijay | CSE |
+-----+-----+
```

Screenshot 6: Execution of Q4 (Full table scan of updated `Employees`) and initiation of Q5 (INNER JOIN).

7. Outer Join Queries: Q5 Result, Q6 (Unassigned Depts), Q7 Start

Demonstrates relational join techniques including filtering unmatched foreign key relationships.

```

mysql>
mysql> -- Q2
mysql> SELECT name, email, role_name
  -> FROM users u
  -> LEFT JOIN roles r
  -> ON u.role_id = r.role_id;
+-----+-----+-----+
| name | email | role_name |
+-----+-----+-----+
| Rahul | rahul@gmail.com | Admin |
| Arjun | arjun@gmail.com | Manager |
| Priya | priya@gmail.com | Employee |
| Sneha | sneha@gmail.com | NULL |
+-----+-----+-----+
4 rows in set (0.00 sec)

mysql>
mysql>
mysql> -- Q3
mysql> SELECT name, role_name
  -> FROM users u
  -> RIGHT JOIN roles r
  -> ON u.role_id = r.role_id;
+-----+-----+
| name | role_name |
+-----+-----+
| Rahul | Admin |
| Arjun | Manager |
| Priya | Employee |
| NULL | Guest |
+-----+-----+
4 rows in set (0.00 sec)

mysql>
mysql>
mysql> -- Q4
mysql> SELECT name, email, role_name
  -> FROM users u
  -> LEFT JOIN roles r
  -> ON u.role_id = r.role_id
  -> UNION

```

Screenshot 7: Q5 results (14 matched rows), Q6 (LEFT JOIN finding employees without departments), and Q7 execution.

8. Q7 Results (RIGHT JOIN) & Q8 (SELF JOIN for Manager Hierarchy)

Maps manager-employee relationships using self-referencing foreign keys and outer joins.

```

-> UNION
-> SELECT name, email, role_name
-> FROM users u
-> RIGHT JOIN roles r
-> ON u.role_id = r.role_id;

```

name	email	role_name
Rahul	rahul@gmail.com	Admin
Arjun	arjun@gmail.com	Manager
Priya	priya@gmail.com	Employee
Sneha	sneha@gmail.com	NULL
NULL	NULL	Guest

5 rows in set (0.01 sec)

```

mysql>
mysql>
mysql> -- Q5
mysql> SELECT e.emp_name AS Employee,
->         m.emp_name AS Manager
-> FROM employees e
-> LEFT JOIN employees m
-> ON e.manager_id = m.emp_id;

```

Employee	Manager
Rahul	NULL
Arjun	Rahul
Priya	Rahul
Sneha	Rahul
Kiran	Rahul
Anil	Arjun
Vijay	Rahul
Meena	Arjun

8 rows in set (0.00 sec)

Screenshot 8: Results for Q7 (Department-to-Employee LEFT JOIN) and Q8 (SELF JOIN on Employees).

9. Aggregation & Subquery Filtering: Q9, Q10 (GROUP BY), Q11 (HAVING)

Performs group aggregation functions to compute department salary averages and filters results using `HAVING`.

```

mysql>
mysql> -- Q6
mysql> SELECT o.order_id,
->         customer_name,
->         city,
->         total_amount
-> FROM customers c
-> JOIN orders o
-> ON c.customer_id = o.customer_id;
+-----+-----+-----+-----+
| order_id | customer_name | city      | total_amount |
+-----+-----+-----+-----+
| 101      | Customer1     | Hyderabad | 50000.00     |
| 102      | Customer2     | Mumbai    | 30000.00     |
| 103      | Customer1     | Hyderabad | 20000.00     |
| 104      | Customer3     | Delhi     | 45000.00     |
| 105      | Customer2     | Mumbai    | 25000.00     |
| 106      | Customer1     | Hyderabad | 15000.00     |
+-----+-----+-----+-----+
6 rows in set (0.00 sec)

mysql>
mysql>
mysql> -- Q7
mysql> SELECT dept_name,
->         COUNT(*),
->         AVG(salary),
->         MAX(salary)
-> FROM employees e
-> JOIN departments d
-> ON e.dept_id = d.dept_id
-> GROUP BY dept_name;
+-----+-----+-----+-----+
| dept_name | COUNT(*) | AVG(salary) | MAX(salary) |
+-----+-----+-----+-----+
| CSE      | 4       | 72500.000000 | 90000.00     |
| ECE      | 2       | 65000.000000 | 70000.00     |
| EEE      | 2       | 47500.000000 | 50000.00     |
+-----+-----+-----+-----+
3 rows in set (0.00 sec)

```

Screenshot 9: Execution of Q9 ('COUNT(*)' total employees), Q10 ('AVG(salary)' grouped by dept), and Q11 ('HAVING' filter > 60,000).

10. Aggregate Extreme Metrics (Q12) & Above Average Subquery (Q13)

Calculates highest/lowest salary thresholds and filters employees earning more than the calculated company average salary.

```

mysql> -- Q8
mysql> SELECT emp_name,
->          salary,
->          dept_name
-> FROM employees e
-> JOIN departments d
-> ON e.dept_id = d.dept_id
-> WHERE salary > 50000
-> AND dept_name = 'CSE';
+-----+-----+-----+
| emp_name | salary | dept_name |
+-----+-----+-----+
| Rahul    | 80000.00 | CSE      |
| Arjun    | 65000.00 | CSE      |
| Priya    | 55000.00 | CSE      |
| Vijay    | 90000.00 | CSE      |
+-----+-----+-----+
4 rows in set (0.00 sec)

mysql>
mysql>
mysql> -- Q9
mysql> SELECT customer_name,
->          SUM(total_amount) AS TotalPurchase
-> FROM customers c
-> JOIN orders o
-> ON c.customer_id = o.customer_id
-> GROUP BY customer_name
-> ORDER BY TotalPurchase DESC;
+-----+-----+
| customer_name | TotalPurchase |
+-----+-----+
| Customer1     | 85000.00      |
| Customer2     | 55000.00      |
| Customer3     | 45000.00      |
+-----+-----+
3 rows in set (0.00 sec)

```

Screenshot 10: Execution of Q12 ('MAX'/'MIN' salary) and Q13 (Subquery filtering employees earning above overall average).

11. Subqueries & Max Salary Filter: Q14 & Q15

Uses nested single-value subqueries to match departmental attributes and retrieve maximum earners.

```

mysql> -- Q10
mysql> SELECT student_name,
->         course_name,
->         faculty_name
-> FROM students s
-> JOIN enrollments e
-> ON s.student_id = e.student_id
-> JOIN courses c
-> ON e.course_id = c.course_id
-> JOIN faculty f
-> ON c.faculty_id = f.faculty_id;
ERROR 1146 (42S02): Table 'dbms_joins_transactions.students' doesn't exist
mysql>
mysql>
mysql> -- Q11
mysql> START TRANSACTION;
Query OK, 0 rows affected (0.00 sec)

mysql>
mysql> INSERT INTO accounts
-> VALUES (104, 'Anil', 25000);
Query OK, 1 row affected (0.00 sec)

mysql>
mysql> COMMIT;
Query OK, 0 rows affected (0.00 sec)

mysql>
mysql>
mysql> -- Q12
mysql> START TRANSACTION;
Query OK, 0 rows affected (0.00 sec)

mysql>
mysql> INSERT INTO accounts
-> VALUES (105, 'Ravi', 10000);
Query OK, 1 row affected (0.00 sec)

mysql>
mysql> ROLLBACK;
Query OK, 0 rows affected (0.00 sec)

```

Screenshot 11: Execution of Q14 (Employees in same department as 'Rahul') and Q15 (Employee with maximum salary).

12. Department Size Filtering: Q16 (`HAVING` clause)

Groups employee counts per department and returns departments exceeding size thresholds.

```
mysql> -- Q13
mysql> START TRANSACTION;
Query OK, 0 rows affected (0.00 sec)

mysql>
mysql> UPDATE accounts
  -> SET balance = balance - 5000
  -> WHERE account_no = 101;
Query OK, 1 row affected (0.00 sec)
Rows matched: 1  Changed: 1  Warnings: 0

mysql>
mysql> UPDATE accounts
  -> SET balance = balance + 5000
  -> WHERE account_no = 102;
Query OK, 1 row affected (0.00 sec)
Rows matched: 1  Changed: 1  Warnings: 0

mysql>
mysql> COMMIT;
Query OK, 0 rows affected (0.00 sec)

mysql>
mysql>
mysql> -- Q14
mysql> START TRANSACTION;
Query OK, 0 rows affected (0.00 sec)

mysql>
mysql> UPDATE accounts
  -> SET balance = balance - 3000
  -> WHERE account_no = 101;
Query OK, 1 row affected (0.00 sec)
Rows matched: 1  Changed: 1  Warnings: 0

mysql>
mysql> UPDATE accounts
  -> SET balance = balance + 3000
```

Screenshot 12: Execution of Q16 (Departments with employee count greater than 5).

13. Window Functions & Outer Join Filtering: Q18 & Q19

Demonstrates window functions for intra-partition metric comparisons and outer joins to identify inactive customers.

```

+-----+-----+
| dept_name | Average_Salary |
+-----+-----+
| CSE       | 82500.000000   |
+-----+-----+
1 row in set (0.00 sec)

mysql>
mysql> -- Q12
mysql> SELECT MAX(salary) AS Highest_Salary,
-> MIN(salary) AS Lowest_Salary
-> FROM Employees;
+-----+-----+
| Highest_Salary | Lowest_Salary |
+-----+-----+
| 104500.00     | 30000.00     |
+-----+-----+
1 row in set (0.00 sec)

mysql>
mysql> -- Q13
mysql> SELECT emp_name, salary
-> FROM Employees
-> WHERE salary > (
-> SELECT AVG(salary)
-> FROM Employees
-> );
+-----+-----+
| emp_name | salary |
+-----+-----+
| Rahul   | 88000.00 |
| Arjun   | 71500.00 |
| Kiran   | 70000.00 |
| Vijay   | 99000.00 |
| Ravi    | 104500.00 |
| Amit    | 79200.00 |
| Divya   | 74800.00 |
+-----+-----+
7 rows in set (0.00 sec)

```

Screenshot 13: Execution of Q18 (Window function `PARTITION BY` for departmental average salary comparison) and Q19 (Customers without orders).

14. Top Sold Products & Max Department Size Subquery: Q20 & Q21

Finds top-performing product entities (`Keyboard` with 13 total sold) and calculates maximum department size using nested aggregations.

```

    -> WHERE account_no = 999;
Query OK, 0 rows affected (0.00 sec)
Rows matched: 0  Changed: 0  Warnings: 0

mysql>
mysql> ROLLBACK;
Query OK, 0 rows affected (0.00 sec)

mysql>
mysql>
mysql> -- Q15
mysql> START TRANSACTION;
Query OK, 0 rows affected (0.00 sec)

mysql>
mysql> UPDATE accounts
    -> SET balance = balance + 2000
    -> WHERE account_no = 101;
Query OK, 1 row affected (0.00 sec)
Rows matched: 1  Changed: 1  Warnings: 0

mysql>
mysql> SAVEPOINT s1;
Query OK, 0 rows affected (0.00 sec)

mysql>
mysql> UPDATE accounts
    -> SET balance = balance - 1000
    -> WHERE account_no = 101;
Query OK, 1 row affected (0.00 sec)
Rows matched: 1  Changed: 1  Warnings: 0

mysql>
mysql> ROLLBACK TO s1;
Query OK, 0 rows affected (0.00 sec)

mysql>
mysql> COMMIT;
Query OK, 0 rows affected (0.00 sec)

```

Screenshot 14: Execution of Q20 (Top selling product by quantity) and Q21 (Department with maximum employee count via nested subquery).

15. Analytical Window Functions: Q21, Q22 (Running Sales), Q23 (Customer Spend Ranking)

Computes monthly cumulative revenue using `SUM() OVER()` and ranks customer total purchase volume using the `RANK()` window function.

```

mysql> -- Create Database
mysql> DROP DATABASE IF EXISTS DBMS_Transactions_Part2;
Query OK, 7 rows affected (0.13 sec)

mysql> CREATE DATABASE DBMS_Transactions_Part2;
Query OK, 1 row affected (0.01 sec)

mysql> USE DBMS_Transactions_Part2;
Database changed
mysql>
mysql> -- Create Departments Table
mysql> CREATE TABLE Departments (
    ->     dept_id INT PRIMARY KEY,
    ->     dept_name VARCHAR(50)
    -> );
Query OK, 0 rows affected (0.03 sec)

mysql>
mysql> -- Create Employees Table
mysql> CREATE TABLE Employees (
    ->     emp_id INT PRIMARY KEY,
    ->     emp_name VARCHAR(50),
    ->     dept_id INT,
    ->     manager_id INT,
    ->     salary DECIMAL(10,2),
    ->     email VARCHAR(100)
    -> );
Query OK, 0 rows affected (0.03 sec)

mysql>
mysql> -- Create Projects Table
mysql> CREATE TABLE Projects (
    ->     project_id INT PRIMARY KEY,
    ->     project_name VARCHAR(100),
    ->     emp_id INT
    -> );
Query OK, 0 rows affected (0.03 sec)

mysql>

```

Screenshot 15: Execution of Q21 (Max Dept Count), Q22 (Monthly Sales & Running Sales OVER Window), and Q23 (Customer Spend Ranking via RANK()).

16. Advanced Self Joins & Duplicate Detection: Q24 & Q25

Identifies employees out-earning their direct managers ('Vijay' and 'Ravi') and checks for email uniqueness constraint violations.

```
mysql> -- Create Customers Table
mysql> CREATE TABLE Customers (
  ->     customer_id INT PRIMARY KEY,
  ->     customer_name VARCHAR(50),
  ->     email VARCHAR(100)
  -> );
Query OK, 0 rows affected (0.04 sec)

mysql>
mysql> -- Create Products Table
mysql> CREATE TABLE Products (
  ->     product_id INT PRIMARY KEY,
  ->     product_name VARCHAR(100),
  ->     price DECIMAL(10,2)
  -> );
Query OK, 0 rows affected (0.03 sec)

mysql>
mysql> -- Create Orders Table
mysql> CREATE TABLE Orders (
  ->     order_id INT PRIMARY KEY,
  ->     customer_id INT,
  ->     order_date DATE
  -> );
Query OK, 0 rows affected (0.02 sec)

mysql>
mysql> -- Create Order_Details Table
mysql> CREATE TABLE Order_Details (
  ->     order_id INT,
  ->     product_id INT,
  ->     quantity INT,
  ->     PRIMARY KEY (order_id, product_id)
  -> );
Query OK, 0 rows affected (0.01 sec)
```

Screenshot 16: Execution of Q24 (SELF JOIN filtering employees earning more than their manager) and Q25 (Duplicate email check via HAVING COUNT(*) > 1).